МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«Пермский государственный национальный исследовательский университет»

Институт компьютерных наук и технологий

ОТЧЁТ

по лабораторной работе №2
по дисциплине «Язык программирования C++»

«Стандартная библиотека шаблонов STL: контейнеры, итераторы и алгоритмы»

Вариант 3 (задачи 1.1, 2.8, 3.6, 4.9, 5.3, 6.3, 7.13, 8.2, 9.2, 10.3)

Выполнил:
студент группы ИТ-3-2026
Денисов Кирилл Ильич
_____ «___» _____ 2026 г.

Проверила:
преподаватель
Ракина Валерия Денисовна
Оценка: _____
_____ «___» _____ 2026 г.

Пермь 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ..... 3
1 Постановка задачи..... 4
2 Структура программы..... 5
2.1 Состав проекта..... 5
2.2 Интерфейс программы..... 5
2.3 Ввод с клавиатуры..... 6
2.4 Вывод контейнеров..... 7
2.5 Соблюдение Google C++ Style Guide..... 8
3 Задание 1. Заполнение и доступ к элементам (задача 1.1)..... 8
4 Задание 2. Вставка элементов (задача 2.8)..... 9
5 Задание 3. Удаление элементов (задача 3.6)..... 10
6 Задание 4. Итераторы и алгоритмы (задача 4.9)..... 11
7 Задание 5. Алгоритмы поиска (задача 5.3)..... 12
8 Задание 6. Базовые модифицирующие алгоритмы (задача 6.3)..... 13
9 Задание 7. Сортировка и слияние (задача 7.13)..... 14
10 Задание 8. Численные алгоритмы (задача 8.2)..... 15
11 Задание 9. Множества (задача 9.2)..... 16
12 Задание 10. Отображения (задача 10.3)..... 17
13 Результаты тестирования..... 19
ЗАКЛЮЧЕНИЕ..... 21
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ..... 22
ПРИЛОЖЕНИЕ А Исходный код программы..... 23

ВВЕДЕНИЕ

Лабораторная работа №2 по дисциплине «Язык программирования C++» посвящена стандартной библиотеке шаблонов STL: последовательным контейнерам `vector`, `deque` и `list`, ассоциативным контейнерам `multiset` и `multimap`, итераторам (в том числе обратным и итераторам ввода-вывода) и обобщённым алгоритмам из заголовков `<algorithm>` и `<numeric>`.

Цель работы – получить практические навыки работы с контейнерами и алгоритмами STL: заполнять контейнеры с помощью конструктора от пары итераторов, выводить их содержимое через итераторы, применять алгоритмы вместо ручных циклов и записывать функциональные объекты в виде лямбда-выражений.

Для достижения цели решались следующие задачи:

– реализовать по одной задаче из каждого из десяти заданий согласно варианту 3: 1.1, 2.8, 3.6, 4.9, 5.3, 6.3, 7.13, 8.2, 9.2 и 10.3;

– оформить решение каждой задачи в виде отдельной функции, а в `main` организовать дружественное текстовое меню выбора задания;

– реализовать проверку ввода: повторный запрос значения при ошибке и корректное завершение при закрытии потока ввода;

– оформить исходный код в соответствии с руководством по стилю Google C++ Style Guide [1];

– протестировать функции на примерах из условия и на граничных случаях.

Программа написана на языке C++ [3] (стандарт C++20) и собирается любым компилятором, который его поддерживает. Дополнительно код проверялся компилятором g++ с ключами `-std=c++20 -Wall -Wextra -pedantic -Wconversion` (без предупреждений) и санитайзерами AddressSanitizer и UndefinedBehaviorSanitizer (ошибок не выявлено). Отчёт оформлен по ГОСТ 7.32–2017 [2].

1 Постановка задачи

Решение всех задач необходимо оформить в виде функций, а в главной функции `main` организовать их вызов с дружественным интерфейсом. Если исходные данные вводятся с клавиатуры, требуется проверять ввод.

Для заданий 1–3 заполнять контейнер нужно с помощью итератора и конструктора соответствующего контейнера, а выводить элементы – через итератор (в обратном порядке – через обратные итераторы `rbegin` и `rend`). Задание 4 выполняется без контейнеров с помощью итераторов `istream_iterator` и `ostream_iterator`. Если алгоритму нужен функциональный объект, используется лямбда-выражение. Если в заданиях 5–10 тип элементов не указан, элементами считаются целые числа.

По варианту 3 из каждого задания решается одна задача (таблица 1).

Таблица 1 – Задачи варианта 3 и реализующие их функции

Задание	№	Название задачи	Функция
1. Заполнение и доступ	1	Вектор из файла	`MakeFileVector`
2. Вставка элементов	8	Вставка −1 в деке	`InsertMinusOne`
3. Удаление элементов	6	Перенос половины списка	`InsertHalfList`
4. Итераторы и алгоритмы	9	Буквы в файл	`WriteLatineInFile`
5. Алгоритмы поиска	3	Удаление нулей	`DeleteZero`
6. Модифицирующие алгоритмы	3	Заполнение дека	`InsertNumbersInDeq`
7. Сортировка и слияние	13	Слияние по знаку	`SortVectorBySign`
8. Численные алгоритмы	2	Пары соседних слов	`MakeWordPairs`
9. Множества	2	Векторы с элементами V0	`CountVectorsWithAll`
10. Отображения	3	Группировка по цифре	`GroupByLastDigit`

Сигнатуры функций выбраны так, чтобы их можно было проверять без ввода с клавиатуры: данные передаются в функцию параметрами, а контейнеры, которые изменяются по условию, – по ссылке. Имена записаны с заглавной буквы по правилам Google C++ Style Guide [1].

2 Структура программы

2.1 Состав проекта

Проект состоит из трёх файлов (код функций заданий – в разделах 3–12, файлы `Methods.h` и `main.cpp` – в приложении А):

– `Methods.h` – объявления функций с комментариями и шаблон `PrintContainer`;

– `Methods.cpp` – реализация десяти функций заданий, функций ввода с проверкой и функций `Task1()`–`Task10()`, которые запрашивают данные и вызывают функции заданий;

– `main.cpp` – функция `Menu` с меню выбора задания и функция `main`.

Свой класс вектора не создавался: везде используется готовый `std::vector`, а остальные контейнеры тоже взяты из стандартной библиотеки.

2.2 Интерфейс программы

Меню одноуровневое: пользователь сразу выбирает номер задания от 1 до 10, а пункт 0 завершает программу. Выбранная функция `TaskN()` запрашивает данные, вызывает функцию решения и выводит результат, после чего снова показывается меню. Пример функции `Task2` приведён в листинге 1, работа меню и обработка неверного ввода – на рисунке 1.

Листинг 1 – Функция Task2

```
// Задание второе
void Task2() {
    std::deque<int> deq = ReadDeque();
    std::cout << "Исходный дек :";
    PrintContainer(deq);
    InsertMinusOne(deq);
    std::cout << "Результат :";
    PrintContainer(deq);
}
```

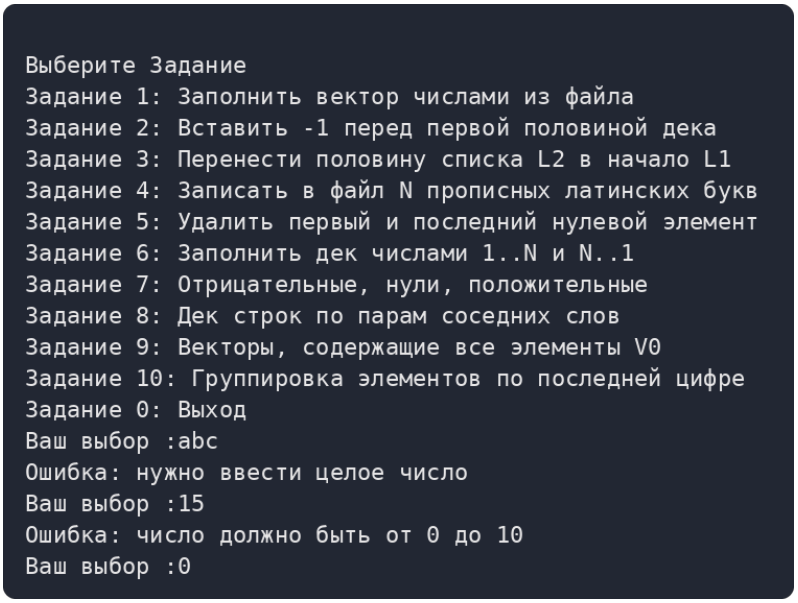


Рисунок 1 – Меню программы и обработка неверного ввода

2.3 Ввод с клавиатуры

Ввод организован в три уровня (листинг 2). Нижний уровень – функция `ReadLine`: она выводит приглашение и читает строку целиком функцией `std::getline`. Если поток ввода закрыт (Ctrl+D), выводится сообщение «Ввод завершён» и программа завершается. Средний уровень – `ReadInt`: строка помещается в `std::istringstream`, из неё извлекается число, после чего делается попытка прочитать ещё один символ. Если символ нашёлся, в строке есть лишнее, и ввод отклоняется. Поэтому строки «abc», «4x» и «5,25» не принимаются. Верхний уровень – `ReadIntInRange`: она повторяет запрос, пока число не попадёт в заданный отрезок.

Листинг 2 – Функции ввода с клавиатуры

```
// Если поток ввода закрыт, завершает программу
std::string ReadLine(const std::string& message) {
    std::cout << message;
    std::string line;
    if (!std::getline(std::cin, line)) {
        std::cout << "Ввод завершён \n";
        std::exit(0);
    }
    return line;
}

// Строка корректна, если в ней только целое число без лишних символов
int ReadInt(const std::string& message) {
    while (true) {
        std::istringstream stream(ReadLine(message));
        int value = 0;
        char extra = '\0';
        if (stream >> value && !(stream >> extra)) {
            return value;
        }
        std::cout << "Ошибка: нужно ввести целое число \n";
    }
}

// Повторяет ввод, пока число не попадёт в отрезок от min_value до max_value
int ReadIntInRange(const std::string& message, int min_value, int max_value) {
    while (true) {
        int value = ReadInt(message);
        if (value >= min_value && value <= max_value) {
            return value;
        }
        std::cout << "Ошибка: число должно быть от " << min_value << " до "
                            << max_value << " \n";
    }
}
```

Для заданий написаны специальные функции ввода: `ReadDeque` и `ReadList` проверяют чётность количества элементов, `ReadThirds` – кратность трём и возрастание чисел внутри каждой трети, `ReadWords` – что слово состоит только из латинских букв, `ReadVectorAtLeast` и `ReadVectorSet` – минимальный размер векторов. Во всех них числа сначала читаются во временный `std::vector`, а затем контейнер создаётся конструктором от пары итераторов, например `std::deque<int>(number.begin(), number.end())`, как требует условие.

2.4 Вывод контейнеров

Элементы всех контейнеров выводит один шаблон `PrintContainer` (листинг 3). Он перебирает контейнер через `const_iterator` и подходит для `vector`, `deque`, `list`, `set` и `multiset`: каждому из них нужны только `begin`, `end` и разыменование итератора. Шаблон определён в заголовочном файле,

потому что компилятору нужен его полный текст в каждом месте использования. Для `multimap` шаблон не подходит, так как элемент отображения – пара, поэтому для него написана отдельная функция `PrintMultimap`.

Листинг 3 – Шаблон PrintContainer

```
template <typename Container>
void PrintContainer(const Container& container) {
    for (typename Container::const_iterator it = container.begin();
              it != container.end(); ++it) {
        std::cout << *it << ' ';
    }
    std::cout << "\n";
}
```

2.5 Соблюдение Google C++ Style Guide

При оформлении кода применялись правила руководства [1]:

– имена функций записываются в стиле PascalCase (`InsertMinusOne`, `ReadIntInRange`), имена переменных и параметров – строчными буквами через подчёркивание (`size_deq`, `first_zero`);

– длина строки не превышает 80 символов (табуляция считается за 4 знака), открывающая скобка стоит в конце строки;

– защита заголовка называется `LAB2_METHODS_H_`, директивы `#include` расположены в алфавитном порядке, каждый подключаемый заголовок действительно используется;

– перед объявлением каждой функции в заголовке есть комментарий о её назначении и ограничениях на данные.

Отступление от руководства одно: для отступов применяется обычная табуляция (в руководстве – два пробела).

3 Задание 1. Заполнение и доступ к элементам (задача 1.1)

Условие. Дан текстовый файл с именем `name`, содержащий строковые представления целых чисел. Заполнить вектор V числами из исходного файла и вывести элементы вектора в исходном порядке.

Решение. Файл открывается потоком `std::ifstream`. Вектор создаётся конструктором от пары итераторов: `istream_iterator<int>(in)`

читает числа из потока, а `istream_iterator<int>()` обозначает конец ввода. Первый аргумент заключён в дополнительные скобки, иначе компилятор воспримет запись как объявление функции (наиболее неприятный разбор). Если после чтения поток не достиг конца файла (`!in.eof()`), в файле встретился не-числовой текст, и выводится предупреждение. Для пустого файла выводится сообщение, иначе элементы печатает `PrintContainer`. Сам файл в меню создаётся функцией `CreateNumbersFile`, которая запрашивает имя, количество и значения чисел.

Листинг 4 – Функция MakeFileVector

```
// Задание 1.1 Создать вектор и заполнить числами из файла
void MakeFileVector(const std::string& name) {
    std::ifstream in(name);
    if (!in) {
        std::cout << "Не удалось открыть файл :" << name << "\n";
        return;
    }
    std::vector<int> v((std::istream_iterator<int>(in)),
                                        std::istream_iterator<int>());
    if (!in.eof()) {
        std::cout << "Чтение завершилось не на числе \n";
    }
    if (v.empty()) {
        std::cout << "Файл пуст \n";
        return;
    }
    PrintContainer(v);
}
```

```
Ваш выбор :1
Введите имя файла :numbers.txt
Сколько чисел записать в файл :5
Число 1:12
Число 2:-7
Число 3:0
Число 4:45
Число 5:3
Вектор из файла :
12 -7 0 45 3
```

Рисунок 2 – Создание файла и вывод вектора из него

4 Задание 2. Вставка элементов (задача 2.8)

Условие. Дан дек D с чётным количеством элементов. Вставить перед каждым элементом из первой половины исходного дека число −1. Использовать функцию-член `insert` в цикле с числовым параметром.

Решение. Итератор `it` устанавливается на начало второй половины дека (`begin() + N / 2`). Цикл повторяется `N / 2` раз: `--it` указывает на

очередной элемент первой половины (с конца), а `insert` вставляет перед ним −1 и возвращает итератор на вставленный элемент. Возвращаемое значение обязательно присваивается обратно в `it`: вставка в дек делает все итераторы недействительными. Вставка идёт от конца первой половины к её началу, поэтому ещё не обработанные элементы не смещаются относительно итератора.

Листинг 5 – Функция InsertMinusOne

```
// Задание 2.8 Вставить -1 перед каждым
// эл. дека с четным кол-вом. эл.
void InsertMinusOne(std::deque<int>& deq) {
    const std::size_t size_deq = deq.size();
    std::deque<int>::iterator it = deq.begin() + size_deq / 2;
    for (std::size_t i = 0; i < size_deq / 2; ++i) {
        it = deq.insert(--it, -1);
    }
}
```

```
Ваш выбор :2
Введите четное количество элементов очереди :5
Ошибка: количество должно быть больше нуля и четным
Введите четное количество элементов очереди :6
Число 1:1
Число 2:2
Число 3:3
Число 4:4
Число 5:5
Число 6:6
Исходный дек :1 2 3 4 5 6
Результат :-1 1 -1 2 -1 3 4 5 6
```

Рисунок 3 – Работа функции InsertMinusOne и проверка чётности

5 Задание 3. Удаление элементов (задача 3.6)

Условие. Даны списки L1 и L2; список L2 имеет чётное количество элементов. Переместить первую половину элементов списка L2 в начало списка L1. Использовать один вызов функции-члена `splice`.

Решение. Итератор `middle` устанавливается на начало второй половины L2 функцией `std::advance`: у списка итераторы двунаправленные, поэтому прямой переход `begin() + N / 2` невозможен. Функция-член `splice` принимает позицию вставки в L1, список-источник и полуинтервал [`begin`, `middle`). Она не копирует значения, а перецепляет

узлы списка, поэтому элементы из L2 исчезают, а итераторы на них остаются действительными.

Листинг 6 – Функция InsertHalfList

```
// Задание 3.6 Перенести первую половину списка L2 в начало списка L1
void InsertHalfList(std::list<int>& list_one, std::list<int>& list_two) {
    std::list<int>::iterator middle = list_two.begin();
    std::advance(middle, list_two.size() / 2);
    list_one.splice(list_one.begin(), list_two, list_two.begin(), middle);
}
```

```
Ваш выбор :3
Количество элементов L1: 3
Число 1:1
Число 2:2
Число 3:3
Количество элементов L2: 3
Ошибка: количество должно быть положительным и чётным
Количество элементов L2: 4
Число 1:10
Число 2:20
Число 3:30
Число 4:40
L1 до :1 2 3
L2 до :10 20 30 40
L1 после :10 20 1 2 3
L2 после :30 40
```

Рисунок 4 – Перенос первой половины L2 в начало L1

6 Задание 4. Итераторы и алгоритмы (задача 4.9)

Условие. Дана строка `name` и целое число N ($1 \le N \le 26$). Записать в текстовый файл с именем `name` N начальных прописных букв латинского алфавита. Использовать алгоритм `generate_n`.

Решение. Контейнеры в задании не используются. Алгоритм `generate_n` вызывает функциональный объект N раз и записывает результаты через выходной итератор. Выходным итератором служит `ostream_iterator<char>`, привязанный к файловому потоку. Лямбда-выражение захватывает переменную `letter` по ссылке (`[&letter]`) и при каждом вызове возвращает текущую букву, увеличивая её на единицу: `A`, `B`, `C`, … Допустимость N проверяет `ReadIntInRange`, поэтому выхода за букву `Z` не будет.

Листинг 7 – Функция WriteLatineInFile

```
// Задание 4.9 Записать в файл N латинских букв
void WriteLatineInFile(const std::string& name, int number) {
    std::ofstream out(name);
    if (!out) {
        std::cout << "Ошибка: не удалось создать файл " << name << "\n";
        return;
    }
    char letter = 'A';
    std::generate_n(std::ostream_iterator<char>(out), number,
                                    [&letter]() { return letter++; });
}
```

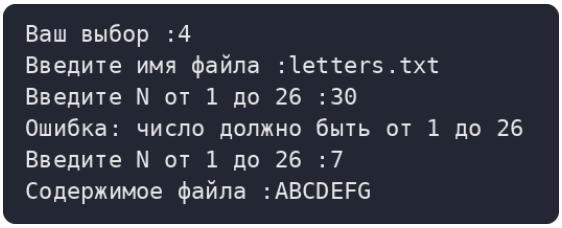


Рисунок 5 – Запись букв в файл и проверка диапазона N

7 Задание 5. Алгоритмы поиска (задача 5.3)

Условие. Дан список L. Удалить первый и последний нулевой элемент списка. Если нулевых элементов нет, список не изменять, если нулевой элемент всего один, удалить только его. Использовать два вызова алгоритма `find` и два вызова функции-члена `erase`.

Решение. Первый нуль ищется `find` по прямым итераторам. Если его нет, функция завершается. Последний нуль ищется вторым вызовом `find` по обратным итераторам `rbegin` и `rend`. Обратный итератор хранит позицию на один элемент правее найденного, поэтому `(++rev).base()` даёт обычный итератор именно на найденный элемент. Сначала удаляется последний нуль, затем – первый, но только если это разные элементы (`first_zero != last_zero`). Для списка удаление одного узла не делает недействительными итераторы на другие, поэтому порядок удаления безопасен. Функцию-член `remove` использовать нельзя: она удалила бы все нули.

Листинг 8 – Функция DeleteZero

```
// Задание 5.3 Удаление первого и последнего нулевого эл. списка
void DeleteZero(std::list<int>& our_list) {
    std::list<int>::iterator first_zero =
            std::find(our_list.begin(), our_list.end(), 0);
    if (first_zero == our_list.end()) {
        return;
    }
    std::list<int>::reverse_iterator rev =
            std::find(our_list.rbegin(), our_list.rend(), 0);
    std::list<int>::iterator last_zero = (++rev).base();
    our_list.erase(last_zero);
    if (first_zero != last_zero) {
        our_list.erase(first_zero);
    }
}
```

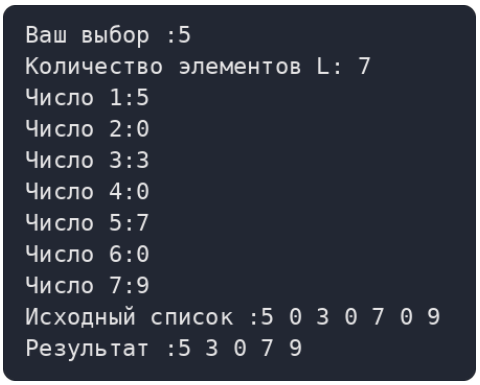


Рисунок 6 – Удаление первого и последнего нуля

8 Задание 6. Базовые модифицирующие алгоритмы (задача 6.3)

Условие. Дано число N (> 0) и дек D, содержащий не менее 2N элементов. Заполнить первые N элементов дека последовательностью чисел 1, 2, …, N, а последние N элементов дека – последовательностью N, N−1, …, 2, 1. Использовать два вызова алгоритма `generate` с одинаковым параметром – функциональным объектом, а также обратные итераторы.

Решение. Лямбда `next` захватывает счётчик `start` по значению и объявлена как `mutable`, поэтому может изменять свою копию: каждый вызов возвращает `++start`. Алгоритм `generate` принимает функциональный объект по значению, поэтому каждому из двух вызовов достаётся своя копия со счётчиком, начинающимся с нуля, и обе последовательности начинаются с единицы. Первый вызов заполняет начало дека прямыми итераторами `[begin, begin + N)`. Второй идёт с конца дека обратными итераторами

`[rbegin, rbegin + N)`: последний элемент получает 1, предпоследний – 2 и так далее, и в прямом порядке получается N, …, 2, 1.

Листинг 9 – Функция InsertNumbersInDeq

```
// Задание 6.3 Перезаписывает N эл. дек с начала и конца
void InsertNumbersInDeq(std::deque<int>& deq, int count) {
    int start = 0;
    auto next = [start]() mutable { return ++start; };
    std::generate(deq.begin(), deq.begin() + count, next);
    std::generate(deq.rbegin(), deq.rbegin() + count, next);
}
```

```
Ваш выбор :6
Введите N (N > 0) :3
Количество элементов дека (не меньше 6) :8
Число 1:10
Число 2:20
Число 3:30
Число 4:40
Число 5:50
Число 6:60
Число 7:70
Число 8:80
Исходный дек :10 20 30 40 50 60 70 80
Результат :1 2 3 40 50 3 2 1
```

Рисунок 7 – Заполнение начала и конца дека

9 Задание 7. Сортировка и слияние (задача 7.13)

Условие. Дан вектор V, количество элементов которого делится на 3. Известно, что каждая треть вектора уже упорядочена по возрастанию. Решить для него задачу 7.7 (расположить элементы так, чтобы сначала шли отрицательные, затем нулевые, а затем положительные, сохранив исходный порядок в каждой группе), используя два вызова алгоритма `inplace_merge` с одним и тем же функциональным объектом.

Решение. Каждая треть упорядочена по возрастанию, значит, внутри неё уже идут отрицательные, затем нули, затем положительные. Если сливать трети по знаку числа, получится нужное расположение. Для этого лямбда `by_sign` сравнивает не сами числа, а их знаки (`Sign` возвращает −1, 0 или 1). `inplace_merge` устойчив: равные по знаку элементы сохраняют исходный порядок, причём элементы первого диапазона идут перед элементами второго. Первый вызов сливает первую и вторую трети, второй – результат с последней третью.

Листинг 10 – Функция SortVectorBySign

```
// Знак числа: -1, 0 или 1.
int Sign(int x) {
    return (x > 0) - (x < 0);
}

// Задание 7.13 сортировка вектора по третям
void SortVectorBySign(std::vector<int>& v) {
    const std::size_t third = v.size() / 3;
    auto by_sign = [](int a, int b) { return Sign(a) < Sign(b); };
    std::inplace_merge(v.begin(), v.begin() + third, v.begin() + 2 * third,
                                        by_sign);
    std::inplace_merge(v.begin(), v.begin() + 2 * third, v.end(), by_sign);
}
```

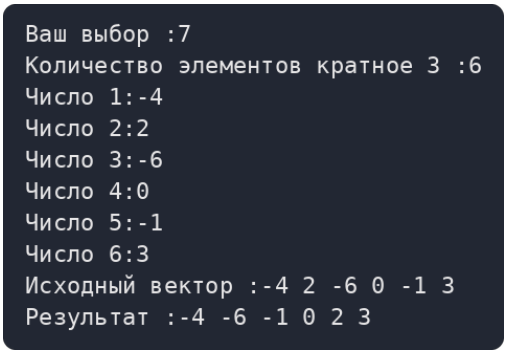


Рисунок 8 – Слияние третей вектора по знаку чисел

10 Задание 8. Численные алгоритмы (задача 8.2)

Условие. Дан список L, элементами которого являются английские слова. Получить дек D со строковыми элементами, каждый из которых строится по паре соседних элементов списка L следующим образом: последняя буква правого элемента пары приписывается справа к первой букве левого. Количество элементов дека D должно быть на 1 меньше количества элементов списка L. Например, для списка ABC, DEF, KLM, XYZ дек должен содержать строки AF, DM, KZ. Использовать алгоритм `adjacent_difference` с итератором вставки и функциональным объектом, а также функцию-член `erase` для дека D.

Решение. Алгоритм `adjacent_difference` вызывает операцию для каждой пары соседних элементов: первым аргументом идёт текущий (правый) элемент, вторым – предыдущий (левый). Лямбда строит строку из первой буквы левого слова и последней буквы правого. Результаты записываются в дек через `back_inserter`. Первым элементом результата алгоритм всегда записывает копию первого слова списка, которая не нужна,

поэтому она удаляется вызовом `erase(pairs.begin())`. Для пустого списка результат пуст, для списка из одного слова после удаления тоже остаётся пустой дек.

Листинг 11 – Функция MakeWordPairs

```
// Задание 8.2 Построить дек строк по парам соседних слов списка
std::deque<std::string> MakeWordPairs(const std::list<std::string>& words) {
    std::deque<std::string> pairs;
    std::adjacent_difference(
            words.begin(), words.end(), std::back_inserter(pairs),
            [](const std::string& right, const std::string& left) {
                return std::string(1, left.front()) + right.back();
            });
    if (!pairs.empty()) {
        pairs.erase(pairs.begin());
    }
    return pairs;
}
```

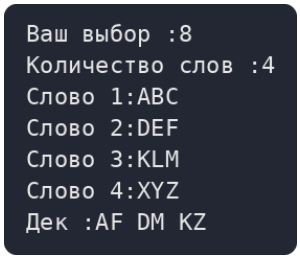


Рисунок 9 – Построение дека по парам соседних слов

11 Задание 9. Множества (задача 9.2)

Условие. Дан вектор V0, целое число N (> 0) и набор векторов V1, …, VN. Известно, что размер V0 не превосходит размера любого из векторов V1, …, VN. Найти количество векторов VI, в которых содержатся все элементы вектора V0 (с учётом их повторений). Использовать алгоритм `includes`, применяя его в цикле к двум мультимножествам (`multiset`), одно из которых создано на основе V0, а другое на очередной итерации содержит элементы очередного вектора VI.

Решение. Мультимножество `set0` строится из V0 один раз до цикла. На каждой итерации из очередного вектора создаётся `current`, и вызывается `includes(current, set0)`. Алгоритм требует упорядоченных диапазонов, а `multiset` хранит элементы упорядоченно, при этом повторы сохраняются. Алгоритм возвращает `true`, если каждый элемент второго диапазона присутствует в первом столько раз, сколько он встречается во втором.

Например, при V0 = (2, 2, 5) вектору (2, 5, 7) не хватает одной двойки, поэтому он не считается. Сложность линейна относительно суммарного размера диапазонов.

Листинг 12 – Функция CountVectorsWithAll

```
// Задание 9.2 Кол-ва. векторов содержащих все элементы V0 с повторениями
int CountVectorsWithAll(
        const std::vector<int>& vec0,
        const std::vector<std::vector<int>>& vectors) {
    const std::multiset<int> set0(vec0.begin(), vec0.end());
    int count = 0;
    for (std::vector<std::vector<int>>::const_iterator it = vectors.begin();
             it != vectors.end(); ++it) {
        const std::multiset<int> current(it->begin(), it->end());
        if (std::includes(current.begin(), current.end(),
                                            set0.begin(), set0.end())) {
            ++count;
        }
    }
    return count;
}
```

```
Ваш выбор :9
Количество элементов V0 (не меньше 1) :3
V0, число 1:2
V0, число 2:2
V0, число 3:5
Введите N (N > 0) :2
Количество элементов V1 (не меньше 3) :4
V1, число 1:2
V1, число 2:5
V1, число 3:2
V1, число 4:7
Количество элементов V2 (не меньше 3) :3
V2, число 1:2
V2, число 2:5
V2, число 3:7
Подходящих векторов :1
```

Рисунок 10 – Подсчёт векторов, содержащих все элементы V0

12 Задание 10. Отображения (задача 10.3)

Условие. Дан вектор V. Выполнить группировку элементов вектора V, используя в качестве ключа группировки последнюю цифру элемента: в одну группу должны входить все элементы вектора V, оканчивающиеся на одну и ту же цифру (сгруппированные элементы должны располагаться в том же порядке, что и в исходном векторе). Результат представить в виде мультиотображения `multimap<int, int>`, ключами которого являются последние цифры, а значениями – элементы вектора. Вывести отображение

(сначала ключ, затем элемент вектора). Для перебора контейнеров использовать циклы с параметрами-итераторами.

Решение. Вектор перебирается циклом с итератором, и каждый элемент вставляется в мультиотображение парой (последняя цифра, элемент). Последняя цифра – это `abs(x % 10)`: для отрицательных чисел остаток от деления в C++ отрицателен, поэтому берётся модуль. Мультиотображение допускает повторяющиеся ключи и упорядочивает пары по ключу, а равные ключи хранит в порядке вставки, поэтому порядок элементов внутри группы совпадает с исходным. Вывод выполняет `PrintMultimap` циклом с итератором.

Листинг 13 – Функция GroupByLastDigit

```
// Вывод мультиотображения ключ затем значение
void PrintMultimap(const std::multimap<int, int>& groups) {
    for (std::multimap<int, int>::const_iterator it = groups.begin();
             it != groups.end(); ++it) {
        std::cout << it->first << ' ' << it->second << "\n";
    }
}

// Задание 10.3 Группировка эл. вектора по последней цифре
std::multimap<int, int> GroupByLastDigit(const std::vector<int>& vec) {
    std::multimap<int, int> groups;
    for (std::vector<int>::const_iterator it = vec.begin(); it != vec.end();
             ++it) {
        groups.insert(std::make_pair(std::abs(*it % 10), *it));
    }
    return groups;
}
```

```
Ваш выбор :10
Количество элементов V (не меньше 1) :8
V, число 1:21
V, число 2:-13
V, число 3:5
V, число 4:33
V, число 5:40
V, число 6:7
V, число 7:-3
V, число 8:10
Группы (ключ и число) :
0 40
0 10
1 21
3 -13
3 33
3 -3
5 5
7 7
```

Рисунок 11 – Группировка элементов вектора по последней цифре

13 Результаты тестирования

Все десять функций проверены автотестами на примерах из условия и на граничных случаях: пустые и одноэлементные контейнеры, отсутствие нулей, единственный нуль, отрицательные числа, повторяющиеся элементы (таблица 2). Тестовая программа вызывает функции без ввода с клавиатуры и сравнивает полученный результат с заранее вычисленным. Выполнено 38 проверок, все пройдены. Дополнительно тесты запускались с санитайзерами AddressSanitizer и UndefinedBehaviorSanitizer, ошибок работы с памятью не выявлено.

Таблица 2 – Результаты тестирования функций заданий 1–10

№	Входные данные	Ожидается	Получено	Итог
1	файл «12 -7 0 45 3»	12 -7 0 45 3	12 -7 0 45 3	пройден
1	файл «7»	7	7	пройден
1	пустой файл	сообщение «Файл пуст»	сообщение «Файл пуст»	пройден
1	файла нет	сообщение «Не удалось открыть файл»	сообщение «Не удалось открыть файл»	пройден
2	D=[1,2,3,4]	[-1,1,-1,2,3,4]	[-1,1,-1,2,3,4]	пройден
2	D=[5,6]	[-1,5,6]	[-1,5,6]	пройден
2	D=[1,2,3,4,5,6]	[-1,1,-1,2,-1,3,4,5,6]	[-1,1,-1,2,-1,3,4,5,6]	пройден
2	D=[]	[]	[]	пройден
3	L1=[1,2,3], L2=[10,20,30,40]	L1=[10,20,1,2,3], L2=[30,40]	L1=[10,20,1,2,3], L2=[30,40]	пройден
3	L1=[], L2=[7,8]	L1=[7], L2=[8]	L1=[7], L2=[8]	пройден
3	L1=[1], L2=[]	L1=[1], L2=[]	L1=[1], L2=[]	пройден
4	N=5	ABCDE	ABCDE	пройден
4	N=1	A	A	пройден
4	N=26	ABCDEFGHIJKLMNOPQRSTUVWXYZ	ABCDEFGHIJKLMNOPQRSTUVWXYZ	пройден
5	L=[5,0,3,0,7,0,9]	[5,3,0,7,9]	[5,3,0,7,9]	пройден
5	L=[1,2,3] (нулей нет)	[1,2,3]	[1,2,3]	пройден
5	L=[1,0,2] (один нуль)	[1,2]	[1,2]	пройден
5	L=[0,0]	[]	[]	пройден
5	L=[0,4,5,0]	[4,5]	[4,5]	пройден
6	N=3, D=[10..80]	[1,2,3,40,50,3,2,1]	[1,2,3,40,50,3,2,1]	пройден
6	N=1, D=[9,9]	[1,1]	[1,1]	пройден
6	N=2, D=[0,0,0,0]	[1,2,2,1]	[1,2,2,1]	пройден
7	V=[-4,2,-6,0,-1,3]	[-4,-6,-1,0,2,3]	[-4,-6,-1,0,2,3]	пройден
7	V=[1,2,3,4,5,6]	[1,2,3,4,5,6]	[1,2,3,4,5,6]	пройден
7	V — только отрицательные	[-3,-2,-1,-6,-5,-4,-9,-8,-7]	[-3,-2,-1,-6,-5,-4,-9,-8,-7]	пройден
7	V=[-2,0,5,-3,0,4,-1,0,6]	[-2,-3,-1,0,0,0,5,4,6]	[-2,-3,-1,0,0,0,5,4,6]	пройден

№	Входные данные	Ожидается	Получено	Итог
8	L=[ABC,DEF,KLM,XYZ]	[AF,DM,KZ]	[AF,DM,KZ]	пройден
8	L=[AB,CD]	[AD]	[AD]	пройден
8	L=[HELLO]	[]	[]	пройден
8	L=[]	[]	[]	пройден
9	V0=[2,2,5]; V1..V3	2	2	пройден
9	V0=[1]; [1],[2],[3,1]	2	2	пройден
9	V0=[4,4]; [4,5],[5,5]	0	0	пройден
9	V0=[3]; [3,3,3]	1	1	пройден
10	V=[21,-13,5,33,40,7,-3,10]	{0:40,0:10,1:21,3:-13,3:33,3:-3,5:5,7:7}	{0:40,0:10,1:21,3:-13,3:33,3:-3,5:5,7:7}	пройден
10	V=[11,21,31]	{1:11,1:21,1:31}	{1:11,1:21,1:31}	пройден
10	V=[0]	{0:0}	{0:0}	пройден
10	V=[-10,10,-5]	{0:-10,0:10,5:-5}	{0:-10,0:10,5:-5}	пройден

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы реализованы десять функций по задачам варианта 3, по одной из каждого задания: работа с файлом через `istream_iterator`, вставка в дек и удаление из списка, запись в файл через `generate_n` и `ostream_iterator`, поиск и удаление нулей прямыми и обратными итераторами, заполнение дека алгоритмом `generate`, слияние алгоритмом `inplace_merge`, построение дека алгоритмом `adjacent_difference`, подсчёт векторов алгоритмом `includes` и группировка в `multimap`. Все функции вызываются из меню, ввод с клавиатуры проверяется: некорректные строки отклоняются, значения запрашиваются повторно, выход за допустимые границы контролируется.

Функции проверены 38 автотестами, все они пройдены, а проверка санитайзерами не выявила ошибок работы с памятью. При решении пришлось учесть особенности STL: вставка в `deque` делает недействительными все итераторы, поэтому итератор присваивается заново; обратный итератор указывает на элемент левее того, на который указывает его `base()`; `generate` копирует функциональный объект, поэтому два вызова с одной лямбдой начинают счёт заново; результат `adjacent_difference` содержит лишний первый элемент.

Исходный код оформлен по руководству Google C++ Style Guide: единые правила именования, длина строк до 80 символов, комментарии к объявлениям и защита заголовка; единственное отступление – табуляция вместо двух пробелов. В результате работы закреплены навыки использования контейнеров, итераторов и алгоритмов STL, а также лямбда-выражений.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Google C++ Style Guide : [сайт]. – URL: https://google.github.io/styleguide/cppguide.html (дата обращения: 21.10.2026). – Текст : электронный.

2. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления : национальный стандарт Российской Федерации : издание официальное : введён впервые 2018-07-01. – Москва : Стандартинформ, 2017. – 32 с. – Текст : непосредственный.

3. Страуструп, Б. Программирование: принципы и практика с использованием C++ / Б. Страуструп ; пер. с англ. – 2-е изд. – Москва : Вильямс, 2016. – 1328 с. – Текст : непосредственный.

4. Справочник по C++ : [сайт] // cppreference.com. – URL: https://ru.cppreference.com/w/cpp (дата обращения: 21.10.2026). – Текст : электронный.

5. Йосаттис, Н. Стандартная библиотека C++: справочное руководство / Н. Йосаттис ; пер. с англ. – 2-е изд. – Москва : Вильямс, 2014. – 1136 с. – Текст : непосредственный.

ПРИЛОЖЕНИЕ А Исходный код программы

Основные функции из файла `Methods.cpp` показаны в разделах 2–12, здесь приведены заголовочный файл `Methods.h` и главный файл `main.cpp`.

Листинг 14 – Файл Methods.h

```
#ifndef LAB2_METHODS_H_
#define LAB2_METHODS_H_

#include <deque>
#include <iostream>
#include <list>
#include <map>
#include <string>
#include <vector>

// Задание 1 Заполнение и доступ к элементам

// Читает числа из файла name в вектор и выводит их в исходном порядке
// Если файл не открылся или в нём нет чисел, выводит сообщение
void MakeFileVector(const std::string& name);

// Задание 2 Вставка элементов

// Вставляет число -1 перед каждым элементом первой половины дека
// Требуется чётное количество элементов
void InsertMinusOne(std::deque<int>& deq);

// Задание 3 Удаление элементов

// Переносит первую половину списка list_two в начало списка list_one
// Требуется чётное количество элементов в list_two
void InsertHalfList(std::list<int>& list_one, std::list<int>& list_two);

// Задание 4 Итераторы и алгоритмы

// Записывает в файл name первые number прописных латинских букв
// Требуется 1 <= number <= 26
void WriteLatineInFile(const std::string& name, int number);

// Задание 5 Алгоритмы поиска

// Удаляет из списка первый и последний нулевой элемент
// Если нулей нет, список не меняется, если нуль один, удаляется только он
void DeleteZero(std::list<int>& our_list);

// Задание 6 Базовые модифицирующие алгоритмы

// Записывает в первые count элементов дека числа 1, 2, ..., count,
// а в последние count элементов числа count, ..., 2, 1
// Требуется count > 0 и размер дека не меньше 2 * count
void InsertNumbersInDeq(std::deque<int>& deq, int count);

// Задание 7 Сортировка и слияние

// Возвращает знак числа: -1, 0 или 1
int Sign(int x);

// Переставляет элементы вектора: сначала отрицательные, затем нулевые,
// затем положительные, порядок внутри каждой группы сохраняется
// Требуется размер, кратный 3, и порядок по возрастанию внутри каждой трети
void SortVectorBySign(std::vector<int>& v);

// Задание 8 Численные алгоритмы

// Строит дек строк по парам соседних слов списка: к первой букве левого
// слова справа приписывается последняя буква правого слова
// Размер дека на 1 меньше размера списка (для списка из 0 или 1 слова пуст)
std::deque<std::string> MakeWordPairs(const std::list<std::string>& words);
```

```
// Задание 9 Множества

// Возвращает количество векторов набора, в которых содержатся все элементы
// vec0 с учётом повторений
int CountVectorsWithAll(
        const std::vector<int>& vec0,
        const std::vector<std::vector<int>>& vectors);

// Задание 10 Отображения

// Группирует элементы вектора по последней цифре (по модулю), порядок
// элементов внутри группы совпадает с исходным
std::multimap<int, int> GroupByLastDigit(const std::vector<int>& vec);

// Ввод с проверкой и вывод

// Выводит приглашение и считывает строку целиком
std::string ReadLine(const std::string& message);

// Запрашивает целое число до тех пор, пока ввод не будет корректным
int ReadInt(const std::string& message);

// Запрашивает целое число из отрезка [min_value, max_value]
int ReadIntInRange(const std::string& message, int min_value, int max_value);

// Запрашивает положительное чётное количество чисел и сами числа,
// возвращает их в деке
std::deque<int> ReadDeque();

// Запрашивает количество чисел (не меньше min_count) и сами числа,
// возвращает их в деке
std::deque<int> ReadDequeAtLeast(int min_count);

// Запрашивает положительное количество чисел (при need_even чётное)
// и сами числа, возвращает их в списке; title показывается в приглашении
std::list<int> ReadList(const std::string& title, bool need_even);

// Запрашивает положительное количество чисел, кратное 3, и сами числа,
// внутри каждой трети числа вводятся по возрастанию
std::vector<int> ReadThirds();

// Запрашивает количество чисел (не меньше min_count) и сами числа,
// возвращает их в векторе; title показывается в приглашении
std::vector<int> ReadVectorAtLeast(const std::string& title, int min_count);

// Запрашивает count векторов, в каждом не меньше min_size элементов
std::vector<std::vector<int>> ReadVectorSet(int count, int min_size);

// Возвращает true, если слово непустое и состоит только из латинских букв
bool IsEnglishWord(const std::string& word);

// Запрашивает положительное количество слов и сами слова, каждое слово
// должно состоять только из латинских букв, возвращает их в списке
std::list<std::string> ReadWords();

// Выводит элементы любого контейнера через пробел и переводит строку
// Подходит для vector, deque, list, set, multiset и т. д., но не для map
// и multimap (их элементы являются парами)
// Тип элементов должен поддерживать вывод через <<
template <typename Container>
void PrintContainer(const Container& container) {
    for (typename Container::const_iterator it = container.begin();
              it != container.end(); ++it) {
        std::cout << *it << ' ';
    }
    std::cout << "\n";
}

// Выводит мультиотображение: в каждой строке ключ, затем значение
void PrintMultimap(const std::multimap<int, int>& groups);
```

```
// Работа с файлами

// Запрашивает имя файла, количество чисел и сами числа, записывает их
// в файл через пробел
// Возвращает имя файла, при ошибке создания возвращает пустую строку
std::string CreateNumbersFile();

// Выполнение заданий

void Task1();
void Task2();
void Task3();
void Task4();
void Task5();
void Task6();
void Task7();
void Task8();
void Task9();
void Task10();

#endif  // LAB2_METHODS_H_
```

Листинг 15 – Файл main.cpp

```
#include <iostream>

#include "Methods.h"

// Выбор задания
void Menu() {
    while (true) {
        std::cout << "\nВыберите Задание \n";
        std::cout << "Задание 1: Заполнить вектор числами из файла \n";
        std::cout << "Задание 2: Вставить -1 перед первой половиной дека \n";
        std::cout << "Задание 3: Перенести половину списка L2 в начало L1 \n";
        std::cout << "Задание 4: Записать в файл N прописных латинских букв \n";
        std::cout << "Задание 5: Удалить первый и последний нулевой элемент \n";
        std::cout << "Задание 6: Заполнить дек числами 1..N и N..1 \n";
        std::cout << "Задание 7: Отрицательные, нули, положительные \n";
        std::cout << "Задание 8: Дек строк по парам соседних слов \n";
        std::cout << "Задание 9: Векторы, содержащие все элементы V0 \n";
        std::cout << "Задание 10: Группировка элементов по последней цифре \n";
        std::cout << "Задание 0: Выход \n";
        int choise_task = ReadIntInRange("Ваш выбор :", 0, 10);

        switch (choise_task) {
            case 1: {
                Task1();
                break;
            }
            case 2: {
                Task2();
                break;
            }
            case 3: {
                Task3();
                break;
            }
            case 4: {
                Task4();
                break;
            }
            case 5: {
                Task5();
                break;
            }
            case 6: {
                Task6();
                break;
```

```
            }
            case 7: {
                Task7();
                break;
            }
            case 8: {
                Task8();
                break;
            }
            case 9: {
                Task9();
                break;
            }
            case 10: {
                Task10();
                break;
            }
            default: {
                return;
            }
        }
    }
}

int main() {
    Menu();
    return 0;
}
```